LECTURE SERIES · DATABASE SYSTEMS

# SQL Joins & Set Operations

*Practice Queries Over the Students · Courses · Enrollments Schema*


**Md. Rakib Trofder**

Lecturer, Bangladesh Army University of Science & Technology

**Saad Ahmed**

Lecturer, Bangladesh Army University of Science & Technology


Inner & Outer Joins · Self Join · UNION / INTERSECT / EXCEPT · Semi & Anti Join

COMBINING ROWS THAT MATCH

# Inner Joins

Every flavour keeps only rows where the join condition is satisfied on both sides.

### THETA JOIN — ANY CONDITION

```
SELECT s.name, s.age, e.grade
FROM Students s
JOIN Enrollments e ON s.age > 20;
```

### EQUI JOIN — EQUALITY CONDITION

```
SELECT s.name, c.course_name, e.grade
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id;
```

### NATURAL JOIN — AUTO-MATCH ON COMMON COLUMNS

```
-- Matches columns with the same name
SELECT * FROM Students
NATURAL JOIN Enrollments;
```

### CROSS JOIN — CARTESIAN PRODUCT

```
SELECT s.name, c.course_name
FROM Students s
CROSS JOIN Courses c;
```

KEEPING THE UNMATCHED ROWS

# Outer Joins

Outer joins keep rows with no match on the other side, filling the missing columns with NULL.

### LEFT OUTER JOIN

```
SELECT s.student_id, s.name, c.course_name, e.grade
FROM Students s
LEFT JOIN Enrollments e ON s.student_id = e.student_id
LEFT JOIN Courses c ON e.course_id = c.course_id
ORDER BY s.student_id;
```

### RIGHT OUTER JOIN

```
SELECT s.name, c.course_name, e.grade
FROM Students s
RIGHT JOIN Enrollments e ON s.student_id = e.student_id
RIGHT JOIN Courses c ON e.course_id = c.course_id
ORDER BY c.course_id;
```

### ! Full Outer Join — not native in MySQL

```
SELECT s.name, c.course_name, e.grade
FROM Students s
LEFT JOIN Enrollments e ON s.student_id = e.student_id
LEFT JOIN Courses c ON e.course_id = c.course_id
UNION
SELECT s.name, c.course_name, e.grade
FROM Students s
RIGHT JOIN Enrollments e ON s.student_id = e.student_id
RIGHT JOIN Courses c ON e.course_id = c.course_id;
```

*MySQL has no FULL OUTER JOIN keyword — UNION a LEFT JOIN with a RIGHT JOIN to simulate it.*

# Self Join & Set Operations

A self join compares a table to itself; set operations combine two SELECTs with the same column shape.

### SELF JOIN — STUDENTS SHARING A MAJOR

```
SELECT a.name AS student1, b.name AS student2,
       a.major
FROM Students a
JOIN Students b ON a.major = b.major
WHERE a.student_id < b.student_id;
```

### UNION — COMBINE TWO RESULT SETS

```
SELECT name, 'Student' AS role FROM Students
UNION
SELECT name, 'Faculty' AS role FROM Faculty;
```

### INTERSECT — SIMULATED WITH IN

```
SELECT name FROM Students
WHERE student_id IN
  (SELECT student_id FROM Enrollments);
```

### EXCEPT — SIMULATED WITH NOT IN

```
SELECT name FROM Students
WHERE student_id NOT IN
  (SELECT student_id FROM Enrollments);
```

FILTERING WITHOUT DUPLICATING COLUMNS

# Semi Join & Anti Join

EXISTS / NOT EXISTS filter one table by presence (or absence) of a match — without pulling in the other table's columns.

### SEMI JOIN — STUDENTS ENROLLED IN A COURSE

```
SELECT s.*
FROM Students s
WHERE EXISTS (
  SELECT 1 FROM Enrollments e
  WHERE e.student_id = s.student_id
);
```

### ANTI JOIN — STUDENTS WITH NO ENROLLMENTS

```
SELECT s.*
FROM Students s
WHERE NOT EXISTS (
  SELECT 1 FROM Enrollments e
  WHERE e.student_id = s.student_id
);
```

### SEMI JOIN — COURSES WITH AT LEAST ONE STUDENT

```
SELECT c.*
FROM Courses c
WHERE EXISTS (
  SELECT 1 FROM Enrollments e
  WHERE e.course_id = c.course_id
);
```

### ANTI JOIN — COURSES WITH NO ENROLLMENTS

```
SELECT c.*
FROM Courses c
WHERE NOT EXISTS (
  SELECT 1 FROM Enrollments e
  WHERE e.course_id = c.course_id
);
```

# Advanced Join Examples

Real queries chain several joins together, often alongside GROUP BY and aggregate functions.

### FOUR-TABLE JOIN WITH A CONDITION

```sql
SELECT s.name AS student_name,
       c.course_name,
       e.grade,
       d.dept_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
JOIN Departments d ON c.department = d.dept_name;
```

### JOIN + AGGREGATE + GROUP BY

```sql
SELECT s.name,
   COUNT(e.course_id) AS courses_taken,
   ROUND(AVG(CASE
     WHEN e.grade = 'A' THEN 4.0
     WHEN e.grade = 'B+' THEN 3.3
     WHEN e.grade = 'B' THEN 3.0
     ELSE 0
   END), 2) AS estimated_gpa
FROM Students s
LEFT JOIN Enrollments e
  ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
ORDER BY estimated_gpa DESC;
```

SEEING THE NULLS A LEFT JOIN PRODUCES

# Spotting Unmatched Rows

A LEFT JOIN plus a NULL check isolates exactly the rows that had nothing to match.

STUDENTS NOT ENROLLED IN ANYTHING

```
SELECT s.student_id, s.name, e.enrollment_id, e.course_id
FROM Students s
LEFT JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.enrollment_id IS NULL;
```

COURSES WITH NO ENROLLMENTS

```
SELECT c.course_id, c.course_name, e.enrollment_id
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
WHERE e.enrollment_id IS NULL;
```

FULL REPORT — EVERY STUDENT, ENROLLED OR NOT

```
SELECT s.student_id, s.name, s.major,
       COALESCE(c.course_name, 'Not Enrolled') AS course,
       COALESCE(e.grade, 'N/A') AS grade
FROM Students s
LEFT JOIN Enrollments e ON s.student_id = e.student_id
LEFT JOIN Courses c ON e.course_id = c.course_id
ORDER BY s.student_id, e.semester;
```